Session 27

JavaScript Asynchronous Programming: Timers, Callbacks, Promises, and Async/Await

---

What are you going to learn in this session?

In this session, you will learn how JavaScript handles tasks that take time to complete, such as API requests, file operations, timers, and user interactions. Unlike many programming languages, JavaScript executes code on a single thread, so it relies heavily on asynchronous programming techniques to perform long-running operations without freezing the application. You will understand how asynchronous code works and why it is essential in modern web development.

You will begin by exploring timer functions such as `setTimeout()` and `setInterval()`, which allow code to execute after a delay or repeatedly at fixed intervals. From there, you will learn callback functions, which were the original approach for handling asynchronous operations. You will understand how callbacks work, their advantages, and the common problems they introduce when applications become more complex.

Finally, you will learn modern asynchronous programming techniques using Promises and `async/await`. You will explore Promise states, error handling using `then()`, `catch()`, and `finally()`, as well as executing multiple asynchronous tasks with `Promise.all()`. By the end of this session, you will be able to fetch data from APIs, manage asynchronous workflows, handle errors effectively, and write clean, professional JavaScript code using modern async patterns.

Why is this topic important?

Asynchronous programming is one of the most important skills in JavaScript development because almost every modern application depends on external resources. Loading user data, retrieving products from a database, processing payments, uploading files, and communicating with APIs all involve asynchronous operations. Without proper async handling, applications become slow, unresponsive, and difficult to maintain.

From an industry perspective, employers expect developers to understand callbacks, Promises, and `async/await` because these concepts are used daily in frontend and backend development. Frameworks such as React, Next.js, Express.js, and Node.js rely heavily on asynchronous workflows. Interviewers frequently ask questions about Promise behavior, error handling, event loops, and async patterns because they reveal whether a developer can build scalable and reliable applications.

How should you prepare yourself to attend this session?

Before attending this session, you should be comfortable with JavaScript fundamentals such as variables, functions, objects, arrays, loops, and conditional statements. Since asynchronous programming heavily relies on functions, you should understand function declarations, callback functions, parameters, return values, and scope.

Make sure Node.js and VS Code are installed and working correctly. Review JavaScript functions and practice creating small programs using loops and conditional logic. Familiarity with DOM events and API concepts will also help because asynchronous programming is commonly used when responding to user actions and communicating with servers.

What should you do after the session?

After completing this session, practice creating timers using `setTimeout()` and `setInterval()`. Build small programs that simulate delayed actions, countdown timers, and repeated updates. Then move on to callback-based exercises before rewriting the same solutions using Promises and `async/await` to understand the evolution of asynchronous programming techniques.

You should also build mini-projects such as weather applications, quote generators, countdown timers, user data fetchers, and task management applications. For interview preparation, practice explaining the differences between callbacks, Promises, and `async/await`, and write examples that demonstrate proper error handling. The ability to manage asynchronous workflows confidently is highly valued in software engineering interviews.

Class Notes

setTimeout

The `setTimeout()` function executes code once after a specified delay.

Syntax

```javascript
setTimeout(callbackFunction, delayInMilliseconds);
```

Example

```javascript
console.log("Start");

setTimeout(() => {
  console.log("Executed after 2 seconds");
}, 2000);

console.log("End");
```

Output

```
Start
End
Executed after 2 seconds
```

Key Points

- Executes only once
- Delay is measured in milliseconds
- Non-blocking operation

Real-World Uses

- Notifications
- Delayed messages
- Loading screens
- Auto logout warnings

setInterval

The `setInterval()` function repeatedly executes code after a fixed interval.

Syntax

```
JavaScript
setInterval(callbackFunction, interval);
```

Example

```
JavaScript
let count = 0;

setInterval(() => {
  count++;
  console.log(count);
}, 1000);
```

Output

```
None
1
2
3
...
```

Stopping an Interval

```
JavaScript
const timerId = setInterval(() => {
  console.log("Running...");
}, 1000);

clearInterval(timerId);
```

Real-World Uses

- Digital clocks
- Live dashboards

- Polling APIs
- Countdown timers

Callbacks

A callback is a function passed as an argument to another function and executed later.

Example

```JavaScript
function greet(name, callback) {
  console.log(`Hello ${name}`);
  callback();
}

function sayBye() {
  console.log("Goodbye");
}

greet("Rahul", sayBye);
```

Output

```None
Hello Rahul
Goodbye
```

Benefits

- Enables asynchronous programming
- Flexible execution flow

Problem: Callback Hell

```JavaScript
task1(() => {
  task2(() => {
    task3(() => {
      task4(() => {
```

```
        console.log("Done");
      });
    });
  });
});
```

Deep nesting makes code difficult to read and maintain.

Promises

A Promise represents the eventual completion or failure of an asynchronous operation.

Promise States

1. Pending
2. Fulfilled
3. Rejected

Creating a Promise

JavaScript
```
const promise = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("Operation Successful");
  } else {
    reject("Operation Failed");
  }
});
```

Why Promises?

- Cleaner than nested callbacks
- Better error handling
- Easier maintenance

then, catch, finally

These methods handle Promise results.

then()

Runs when a Promise resolves successfully.

```JavaScript
promise.then(result => {
  console.log(result);
});
```

catch()

Runs when a Promise is rejected.

```JavaScript
promise.catch(error => {
  console.error(error);
});
```

finally()

Runs regardless of success or failure.

```JavaScript
promise.finally(() => {
  console.log("Cleanup completed");
});
```

Complete Example

```JavaScript
fetchData()
  .then(data => {
```

```
    console.log(data);
  })
  .catch(error => {
    console.error(error);
  })
  .finally(() => {
    console.log("Request completed");
  });
```

Promise.all

`Promise.all()` executes multiple Promises concurrently and waits until all are resolved.

Example

JavaScript

```
const p1 = Promise.resolve("User");
const p2 = Promise.resolve("Orders");
const p3 = Promise.resolve("Products");

Promise.all([p1, p2, p3])
  .then(results => {
    console.log(results);
  });
```

Output

None

```
["User", "Orders", "Products"]
```

Important Rule

If any Promise rejects, the entire `Promise.all()` operation rejects.

Use Cases

- Loading dashboard data
- Multiple API requests

- Parallel processing

async/await

`async/await` provides a cleaner way to work with Promises.

async Function

```JavaScript
async function greet() {
  return "Hello";
}
```

Returns a Promise automatically.

await Keyword

```JavaScript
async function getData() {
  const result = await Promise.resolve("Data Loaded");

  console.log(result);
}

getData();
```

Output

```None
Data Loaded
```

Benefits

- Cleaner syntax
- Easier debugging
- Readable asynchronous code

Error Handling with async/await

Use `try...catch` blocks.

```JavaScript
async function fetchUser() {
  try {
    const data = await getUser();

    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Why Use try/catch?

- Centralized error handling
- Cleaner code
- Easier debugging

Async Patterns Comparison

JavaScript has evolved through multiple asynchronous patterns.

Callback Pattern

```JavaScript
getData(function(result) {
  console.log(result);
});
```

Pros

- Simple for small tasks

Cons

- Callback hell
- Difficult error handling

Promise Pattern

```JavaScript
getData()
  .then(result => {
    console.log(result);
  })
  .catch(error => {
    console.error(error);
  });
```

Pros

- Better readability
- Structured error handling

Cons

- Chaining can become lengthy

Async/Await Pattern

```JavaScript
async function loadData() {
  try {
    const result = await getData();

    console.log(result);
  } catch (error) {
    console.error(error);
  }
}
```

Pros

- Most readable
- Easy maintenance
- Modern industry standard

Cons

- Requires understanding Promises

Comparison Table

Feature	Callbacks	Promises	Async/Await
Readability	Low	Medium	High
Error Handling	Difficult	Good	Excellent
Nesting Issues	High	Low	Very Low
Industry Usage	Legacy	Common	Most Preferred

Interview Prep Recap

Students should now be able to confidently explain:

- What asynchronous programming is
- Purpose of setTimeout and setInterval
- How callbacks work
- Callback hell problem
- Promise lifecycle and states
- then, catch, and finally methods
- Promise.all execution behavior
- async and await keywords
- Error handling with try/catch
- Comparison between async patterns
- Modern JavaScript asynchronous workflows

Sample Interview Questions

Q1. What is asynchronous programming?

Answer:
A programming approach where tasks can execute without blocking the main execution thread.

Q2. What is the difference between setTimeout and setInterval?

Answer:
`setTimeout()` executes once after a delay, while `setInterval()` executes repeatedly at fixed intervals.

Q3. What is a callback function?

Answer:
A function is passed as an argument to another function and executed later.

Q4. What is callback hell?

Answer:
Deeply nested callback functions that reduce readability and maintainability.

Q5. What are the states of a Promise?

Answer:
Pending, Fulfilled, and Rejected.

Q6. What does Promise.all() do?

Answer:
Executes multiple Promises in parallel and resolves when all complete successfully.

Q7. What is the purpose of catch()?

Answer:
To handle Promise rejections and runtime errors.

Q8. Why is async/await preferred?

Answer:
Because it makes asynchronous code easier to read, write, and maintain.

Q9. Can await be used outside an async function?

Answer:
No, except in environments that support top-level await.

Q10. How do you handle errors with async/await?

Answer:
Using `try...catch` blocks.

Common Beginner Confusions

Confusion:1

setTimeout pauses JavaScript execution.

Reality

It schedules execution and immediately continues running other code.

How to Remember It

Timer schedules work, it does not stop the program.

Confusion:2

setInterval automatically stops.

Reality

It continues running until explicitly cleared.

How to Remember It

Interval repeats forever unless stopped.

Confusion:3

Callbacks are asynchronous by default.

Reality

Callbacks can be synchronous or asynchronous depending on usage.

How to Remember It

Callback means "called later," not necessarily asynchronous.

Confusion:4

Promises execute only when then() is called.

Reality

Promises start executing immediately when created.

How to Remember It

Promise starts now, then() listens later.

Confusion:5

then() handles errors.

Reality

Errors should be handled using catch().

How to Remember It

then = success, catch = failure.

Confusion:6

finally() receives the resolved value.

Reality

finally() runs regardless of outcome and receives no result data.

How to Remember It

finally = cleanup only.

Confusion:7

Promise.all executes Promises one by one.

Reality

Promises run concurrently.

How to Remember It

All means together.

Confusion:8

async makes functions synchronous.

Reality

The function still returns a Promise.

How to Remember It

async improves syntax, not execution model.

Confusion:9

await blocks the entire application.

Reality

It pauses only the async function execution.

How to Remember It

Function waits, application continues.

Confusion:10

Callbacks, Promises, and async/await are completely different technologies.

Reality

Promises build upon callbacks, and async/await builds upon Promises.

How to Remember It

Evolution, not replacement.

Key Takeaways

In this session, you learned how JavaScript handles asynchronous operations using timers, callbacks, Promises, and `async/await`. You explored `setTimeout()` and `setInterval()` for scheduling tasks, understood how callbacks enable deferred execution, and learned why deeply nested callbacks can create maintenance challenges. These concepts form the foundation of asynchronous programming in JavaScript.

You also mastered Promises and their lifecycle, including how to handle successful operations using `then()`, manage failures with `catch()`, and perform cleanup using `finally()`. Additionally, you learned how `Promise.all()` executes multiple asynchronous operations concurrently, making applications faster and more efficient when dealing with multiple data sources.

From an industry and interview perspective, asynchronous programming is a core skill for frontend and backend developers. Modern applications rely heavily on API communication, database operations, file processing, and real-time interactions, all of which require asynchronous workflows. Understanding callbacks, Promises, and `async/await` prepares you for professional development environments and provides a strong foundation for learning advanced topics such as Fetch API, REST APIs, WebSockets, and server-side JavaScript development with Node.js.